NLP 基础横纵分析报告

> 从统计词频到 BERT 再到 GPT——自然语言处理如何在八十年间从"数词数"进化到"理解语义"。

一、一句话定义

自然语言处理（NLP）是让计算机理解、生成和操作人类语言的技术体系。它的核心难题是：**同一个词在不同上下文中的含义完全不同**——"苹果很好吃"和"苹果发布了新手机"中的"苹果"不是一个东西。让机器区分这种差异，是 NLP 八十年演进的核心驱动力。

> 🎯 **读完这篇你能**：理解 NLP 从规则→统计→词向量→预训练 LM→LLM 的五代演进，知道 BERT 和 GPT 的根本区别，能解释为什么 GPT 路线最终胜出。

二、技术背景

2.1 NLP 的核心挑战

自然语言对机器来说，天然是"非结构化"的。一张图片 = 像素矩阵，天然是数值。一段文本 = 一串字符，需要先变成数值才能送入模型。

让文本变成数字——这是 NLP 所有工作的起点。不同的"数字化"方式，定义了不同的 NLP 时代。

2.2 NLP 的五代演进

```
规则时代（1950s-1980s）
  → 统计时代（1990s-2013）：TF-IDF、n-gram、HMM
  → 词向量时代（2013-2018）：Word2Vec、GloVe、FastText
  → 预训练 LM 时代（2018-2020）：BERT、GPT-1、ELMo
  → LLM 时代（2020-）：GPT-3/4/5、Claude、Gemini
```

三、纵向分析

3.1 规则时代（1950s–1980s）—— 手写语法规则

早期的 NLP 基于人工编写的语法规则。比如做翻译：人工写几千条"如果主语是 XXX，动词是 YYY，则翻译为 ZZZ"的规则。

问题：人类语言的规则根本写不完——"我打了他"合法，"他打了我"也合法，"打了我他"不合法但人能理解。规则系统脆弱到换个句式就崩。

3.2 统计时代（1990s–2013）—— 让数据说话

不再手写规则，而是从大量文本中统计规律：

TF–IDF：一个词在一篇文章中出现多次 → 重要。但如果在所有文章中都频繁出现（如"的"）→ 不重要。TF–IDF = 词频 × 逆文档频率，至今仍是文本检索的基石。

n–gram 语言模型：用前 n–1 个词预测下一个词的概率。P(猫 | 喜欢, 吃, 鱼, 的) 从语料库中统计"喜欢吃鱼的"后面出现"猫"的次数。

局限：统计方法看不到"语义"。对于 n–gram 来说，"猫"和"狗"跟"桌子"一样——都只是一个 token ID。它们之间的相似性完全丢失了。

3.3 词向量时代（2013–2018）—— 词语有了"语义空间"

Word2Vec（Mikolov et al., Google, 2013） 是 NLP 进入深度学习时代的分水岭。

核心思想（分布假说）：**一个词的含义由它周围的词决定。** "猫"和"狗"出现在相似的上下文中（"我家养了一只_"，"_在沙发上睡觉"），所以它们的词向量应该接近。

Word2Vec 有两个经典训练方式：

- **CBOW**：用周围词预测中心词
- **Skip–gram**：用中心词预测周围词

训练完的副产品——300–500 维的词向量——带来了 NLP 历史上最著名的语义代数：`king − man + woman ≈ queen`。不是巧合，而是在向量空间中，"皇室"方向减去"男性"方向加上"女性"方向恰好落在"女王"附近。

GloVe（Pennington et al., Stanford, 2014） 结合了全局统计信息（共现矩阵）和局部上下文（滑动窗口）。

FastText（Bojanowski et al., Facebook, 2016） 把词拆成子词（subword），解决了"没见过这个词"的问题——"unhappiness"可以拆成 un + happy + ness，即使模型没见过这个词也能猜到它的意思。

但词向量的根本局限在于：**一个词只有一个固定的向量**。"苹果"在"好吃"和"发布新手机"两个上下文中——向量完全一样。静态词向量无法捕捉这种歧义。

3.4 预训练 LM 时代（2018–2020）—— 上下文嵌入的革命

ELMo（Peters et al., 2018） 第一次做到了"同一个词，不同上下文，不同向量"。它用的是双向 LSTM——从左到右和从右到左分别处理，然后拼接。但 LSTM 的速度和效果都受限。

BERT（Devlin et al., Google, 2018.10） 用 Transformer 替换了 LSTM，引爆了 NLP 的范式革命。

BERT 的训练方式极其巧妙——**掩码语言模型（MLM）**：随机遮住 15% 的 token，让模型根据上下文"填空"。"我 [MASK] 吃苹果" → 模型猜"喜欢"。

关键区别：LSTM 只能从左到右逐字读（单向），BERT 同时看左右两边（双向）——"填空"这个任务天然要求模型理解全局语境。

BERT 发布后横扫了 11 项 NLP 基准。2018–2020 年间，"拿 BERT 微调一下"成为 NLP 所有任务的标准公式。

GPT-1（Radford et al., OpenAI, 2018.06） 选了另一条路——不是填空，而是**自回归**：已知前文，预测下一个词。"我喜欢吃 ___" → 预测"苹果"。

GPT 比 BERT 更接近"真正的语言生成"——因为它生成的 token 又可以作为下一轮的输入，形成完整的文本。

3.5 为什么 GPT 路线最终胜出?

2020 年 GPT-3 的发布宣告了竞争的结果。

- BERT 的优势是双向理解强，但生成能力弱（只能填一个 mask，不能生成完整文本）
- GPT 的优势是生成能力强，随着模型变大，理解能力也追上来了
- GPT-3 的 175B 参数展现了 BERT 类模型无法企及的涌现能力——few-shot learning、代码生成、推理

BERT 没有被废弃——它的双向编码能力被 BGE、E5 等嵌入模型继承，在搜索和 RAG 中作为"文本 → 向量"的编码器继续活跃。但"AI 语言交互"的主流已经锁定在了 GPT 类自回归路线上。

四、核心概念速查

概念	一句话
Tokenization	把文本切成最小单元（词/子词/字符）
词向量	把 token 映射到连续向量空间
静态词向量	一个词只有一种向量表示（Word2Vec/GloVe）
上下文嵌入	同一个词在不同上下文中向量不同（BERT/GPT）
自回归 LM	从左到右逐 token 生成（GPT 系列）
掩码 LM	遮住部分 token 让模型猜（BERT 系列）
编码器-解码器	Encoder 理解输入，Decoder 生成输出（T5/BART）
Few-shot Learning	给几个示例就能做新任务（GPT-3 的核心涌现能力）

五、延伸阅读

- 深度学习基础 — 神经网络、反向传播、CNN/RNN 的前置知识
- LLM 大模型 — GPT/BERT 之后的大模型时代全景
- RAG — 检索增强生成的演进与检索技术的关系

本文是横纵分析系列的第 32 篇报告。定位为基础知识，覆盖 NLP 从规则到 LLM 的五代演进，为后续的大模型技术专题提供入门铺垫。